AutoRH

Fiches de paie, demandes RH et documents signés

Version SANS LIMITES

Comptes, employés et fiches de paie illimités

VERSION 0.2.201

Application Next.js 16 + Prisma + SQLite + Chiffrement AES-256-GCM

Thème : blanc + vert émeraude — Renommée AutoRH

Généré le 14/10/2026

Compatible Windows / Linux / macOS — déploiement local recommandé

Vérification de l'intégrité du ZIP :

Linux/macOS : md5sum <fichier.zip>

Windows (PowerShell) : Get-FileHash <fichier.zip> -Algorithm MD5

Puis : node scripts/verifier-extraction.js

Sommaire

- **1.** Présentation — Version spécifique au ZIP
- **2.** Fonctionnalités essentielles — import, portail, signature, demandes
- **3.** Installation et démarrage — prérequis, lanceurs, secrets
- **4.** Sécurité — chiffrement, authentification, variables sensibles
- **5.** Exploitation et dépannage — commandes, sauvegarde, dépannage
- **6.** Architecture technique — stack, ports, structure dossiers

1. Présentation — Version SANS LIMITES

Cette version d'AutoRH n'impose AUCUN quota sur le nombre de comptes, d'employés ou de fiches de paie. Elle est destinée aux déploiements de production où l'application doit évoluer librement selon les besoins du service RH — par exemple pour gérer plusieurs centaines d'employés et des milliers de fiches de paie. Toutes les fonctionnalités de l'application sont préservées : import PDF/ZIP/Excel, signature électronique, portail employé, demandes administratives, envoi d'e-mails SMTP/IMAP.

1.1. Capacités illimitées

Entité	Limite	Commentaire
Comptes SuperAdmin	Illimité	Bonne pratique : 1-2 comptes max
Comptes IT	Illimité	Bonne pratique : 1-3 comptes max
Comptes RH	Illimité	Bonne pratique : 1-5 comptes selon taille
Employés (accès portail)	Illimité	Testé jusqu'à 1 000+ employés
Fiches de paie importées	Illimité	Testé jusqu'à 240 000 fiches (indexation déterministe)
Demandes administratives	Illimité	Conservation à vie (journalisation)
Documents signés	Illimité	Stockage chiffré sur disque

1.2. Différences avec la version « Avec limites »

Cette version ne contient PAS le module src/lib/limits.ts ni l'endpoint /api/limits. Toutes les routes de création (users, portal-access, payslips/upload) acceptent un nombre illimité d'éléments. L'interface n'affiche pas de badges de quota dans les en-têtes des onglets. Le script scripts/creer-superadmin.js ne vérifie pas le nombre de SuperAdmin existants avant création.

> Si vous souhaitez réactiver les quotas ultérieurement, installez la version « Avec limites » — le schéma de base de données est identique, vous pouvez donc migrer sans perte de données.

1.3. Recommandations de déploiement

- Réservez le rôle SuperAdmin à 1 ou 2 personnes de confiance (administration complète)
- Limitez les comptes IT à 1-3 personnes (diagnostic, configuration SMTP, sauvegardes)
- Comptes RH : selon la taille du service (1-5 typiquement)
- Surveillez l'espace disque : chaque fiche de paie chiffrée occupe ~100-500 Ko
- Sauvegardez régulièrement db/custom.db ET storage/ ET .env (voir section 5.3)

2. Fonctionnalités essentielles

AutoRH couvre l'intégralité du cycle de gestion des fiches de paie et des demandes administratives RH. L'application est organisée en plusieurs onglets accessibles selon le rôle de l'utilisateur connecté (SuperAdmin, RH, IT) : chaque rôle voit uniquement les fonctionnalités pertinentes, conformément à la matrice des droits décrite en annexe. Les employés, quant à eux, accèdent à un portail dédié (port 4001) où ils consultent leurs fiches et déposent leurs demandes.

2.1. Import des fiches de paie

• Formats acceptés : PDF (mono-fiche et multi-fiches), PNG, JPG, JPEG, ZIP (archives imbriquées jusqu'à 2 niveaux), XLSX/XLS (Excel), DOCX (Word)

• Détection automatique : CIN (8 chiffres), nom, prénom, période de paie, net à payer, matricule

• OCR intégré : Tesseract.js avec données fra.traineddata (téléchargées au 1er lancement ou fournies dans ocr-data/)

• Découpage automatique : un PDF multi-pages devient N fiches Payslip (1 par page ou plus), chacune avec son propre CIN détecté

• Indexation déterministe : recherche par CIN en SQL même avec 240 000 fiches (empreinte AES-256-GCM déterministe)

• Détection d'anomalies : noms/prénoms incohérents entre fiches du même CIN signalés automatiquement

2.2. Portail employé (port 4001)

• Connexion sécurisée par CIN + mot de passe (politique d'expiration 15 jours)

• Code OTP optionnel (TOTP standard — Google Authenticator, Microsoft Authenticator, Authy)

• Consultation des fiches de paie signées électroniquement (PAdES ou tampon visuel)

• Dépôt de demandes administratives : congé, autorisation de sortie, attestation de travail, attestation de salaire, parrainage

• Validation à deux étapes : manager → RH (depuis v0.2.168) — refus immédiat si l'un refuse

• Notification automatique par e-mail à chaque étape (employé, RH, manager)

2.3. Signature électronique et tamponnage

• Tampon visuel : image (PNG/JPG), texte, QR code, cachet de certificat — positionnement libre (4 coins + décalages X/Y)

• Signature électronique PAdES : signature cryptographique PKCS#7 avec certificat X.509 (.p12/.pfx)

• Copies signées figées : la première signature d'une fiche est conservée à vie (jamais re-signée)

• Vérification QR code : chaque fiche signée porte un QR code unique permettant sa vérification hors-ligne

2.4. Gestion des demandes administratives

• 5 types : CONGE, AUTORISATION_SORTIE, ATTESTATION_TRAVAIL, ATTESTATION_SALAIRE, PARRAINAGE

• Workflow à deux étapes : décision du manager puis décision du RH (depuis v0.2.168)

• Génération automatique du PDF de réponse signé + tamponné (acceptation) ou e-mail de notification (refus)

• Documents des décideurs (manager + RH) transmis à l'employé en pièces jointes

• Journal complet : toutes les décisions sont tracées dans /api/demandes/journal

2.5. Envoi d'e-mails (SMTP + IMAP)

• Configuration SMTP complète : hôte, port, identifiants, expéditeur — mot de passe chiffré AES-256-GCM en base

• Test SMTP intégré (bouton « Tester la connexion ») avec diagnostic d'erreur précis

- Réception IMAP : interrogation automatique de la boîte RH toutes les 15 secondes pour récupérer les réponses aux demandes
- E-mail automatique à la création d'un compte RH/IT (identifiant + mot de passe initial valable 15 min)
- E-mail automatique aux employés (mot de passe initial du portail, notifications de décision)
- Journal des envois : logs/journal-emails.log trace tous les envois (succès + échecs + réponse SMTP)

3. Installation et démarrage

3.1. Prérequis

L'application AutoRH s'exécute en local sur un poste de travail ou un petit serveur. Les prérequis matériels et logiciels sont volontairement minimes pour permettre un déploiement rapide dans n'importe quel environnement d'entreprise, sans infrastructure cloud ni base de données externe.

- Node.js 20 ou supérieur (téléchargeable sur nodejs.org — version LTS recommandée)
- npm 10 ou supérieur (livré avec Node.js)
- 200 Mo d'espace disque libre (code + dépendances + base SQLite + fiches de paie chiffrées)
- Connexion Internet lors de la première installation (téléchargement des 830 paquets npm) — ensuite l'application fonctionne hors-ligne
- Système d'exploitation : Windows 10/11, Linux (Ubuntu 20.04+), macOS 12+

3.2. Procédure d'installation

L'archive ZIP livrée contient l'intégralité du projet Next.js prêt à l'emploi. La procédure d'installation est identique sur les trois systèmes d'exploitation, à l'exception des lanceurs graphiques (.bat sous Windows, .sh sous Linux/macOS). La première installation prend entre 3 et 5 minutes selon la vitesse de la connexion Internet (téléchargement des dépendances npm) et la puissance de la machine (génération du client Prisma).

Sous Windows (le plus simple)

```
Double-cliquez sur PREPARER-AUTO-RH-SILENCIEUX.bat → installation silencieuse complète (npm install + prisma + vérifications) → un raccourci « AutoRH - Démarrage silencieux » est créé dans le menu Démarrer
```

Sous Linux / macOS

```
chmod +x DEMARRER-TOUT-LINUX-MAC.sh ./DEMARRER-TOUT-LINUX-MAC.sh
```

Installation manuelle (tous OS)

```
cd /chemin/vers/auto-rh-extrait npm install --no-audit --no-fund npx prisma generate npx prisma db push --accept-data-loss npm run dev # serveur développement (port 4000)
```

L'application démarre sur le port 4000 (administration) et 4001 (portail employé). Le pont d'aperçu occupe le port 3000 (désactivable via AUTO_RH_PONT_PREVIEW=0 dans .env).

4. Sécurité

AutoRH a été conçu avec une approche « sécurité par défaut » : toutes les données sensibles sont chiffrées, les sessions sont signées, et l'authentification repose sur des primitives cryptographiques standard (scrypt pour le hachage des mots de passe, AES-256-GCM pour le chiffrement des champs en base et des fichiers stockés, HMAC-SHA256 pour la signature des sessions et des jetons QR). Aucune donnée personnelle ne circule en clair sur

le disque, et les mots de passe ne sont JAMAIS stockés en clair — même pas temporairement.

4.1. Chiffrement des données

- Mots de passe : hachage scrypt (format « scrypt:salt:hash ») — 64 octets de sortie, sel aléatoire de 16 octets par compte
- Fichiers de paie stockés : chiffrement AES-256-GCM avec clé dérivée de AUTORH_STORAGE_KEY via scrypt
- Champs sensibles en base (emails, CIN, mots de passe SMTP) : chiffrement AES-256-GCM à IV aléatoire (recherche impossible) ou déterministe (recherche par égalité possible via empreinte)
- Copies signées des fiches : conservées chiffrées sur disque (storage/copies-signees/), permissions 0600
- Tampons et certificats : stockés chiffrés, jamais renvoyés à l'interface

4.2. Authentification et sessions

- Sessions : cookie httpOnly signé HMAC-SHA256, expiration 15 minutes (session courte) ou 15 jours (mode « se souvenir de moi »)
- Politique de mot de passe : 8 caractères minimum, minuscule + majuscule + chiffre + symbole
- Expiration des mots de passe : tous les 15 jours (politique configurable via PASSWORD_MAX_AGE_DAYS)
- Mot de passe initial des comptes IT/RH : valable 15 minutes uniquement, désactivation automatique sans activation
- Anti force-brute : verrouillage temporaire après 5 échecs consécutifs, déverrouillage par le SuperAdmin
- Journalisation : toutes les connexions (succès + échecs) sont tracées dans logs/auth-journal.jsonl

4.3. Variables d'environnement sensibles (.env)

Le fichier .env (créé à partir de .env.example) contient les secrets de l'application. Il ne doit JAMAIS être commité dans un dépôt git ni partagé par e-mail. Voici les variables critiques :

Variable	Rôle	Obligatoire
DATABASE_URL	Chemin de la base SQLite (ex: file:./db/custom.db)	Oui
AUTH_SECRET	Secret HMAC pour signer les sessions (48+ caractères hex)	Oui en prod
AUTORH_STORAGE_KEY	Clé de chiffrement AES-256 des fichiers stockés (48+ caractères base64)	Oui en prod
LICENSE_HMAC_KEY	Secret HMAC pour valider les clés de licence (essai 15j)	Essai uniquement
LIMIT_MAX_*	Quotas configurables (1 IT, 1 RH, 5 employés, 10 fiches par défaut)	Avec limites

5. Exploitation et dépannage

5.1. Commandes utiles

```
npm run dev # serveur développement (port 4000) npm run build # build production npm run start # serveur production standalone npm run start:portail # portail employé seul (port 4001) npm run db:push # appliquer le schéma Prisma (sans perte) npm run db:reset # réinitialiser la base (POURSUIT TOUT) node scripts/auto-rh.js start # démarrer administration + portail node scripts/auto-rh.js stop # arrêter les deux serveurs node scripts/auto-rh.js status # état des serveurs
```

5.2. Vérification de l'extraction du ZIP

Après extraction du ZIP livré, exécutez le vérificateur d'intégrité pour vous assurer qu'aucun fichier n'est manquant ou corrompu :

```
node scripts/verifier-extraction.js # Code de sortie 0 = extraction complète OK # Code de sortie 2 = fichiers manquants/corrompus (re-extrayez) # Code de sortie 3 = manifeste absent (ZIP trop ancien)
```

5.3. Sauvegarde et restauration

- Base de données : copier db/custom.db (et db/custom.db-journal si présent)
- Fichiers de paie : copier le dossier storage/ (fichiers chiffrés AES-256-GCM)
- Configuration : copier le fichier .env (contient les clés de chiffrement — SANS lui, les données sont IRRÉCUPÉRABLES)
- Sauvegarde automatique Windows : SAUVEGARDER-BD-WINDOWS.bat

5.4. Dépannage courant

Problème	Cause	Solution
« Couldn't find any pages or app directory »	Extraction ZIP incomplète	Re-extrayez TOUT le ZIP, puis node scripts/verifier-extraction.js
Page de connexion en boucle	AUTH_SECRET modifié (sessions invalidées)	Reconnectez-vous (cookies périmés)
« Module @napi-rs/canvas introuvable »	Dépendance native manquante	npm install --no-audit (re-télécharge les binaires)
Fiches de paie illisibles après restauration	AUTORH_STORAGE_KEY différent	Restaurez le .env ORIGINAL (clé de chiffrement perdue = donnée
« Base introuvable »	DATABASE_URL incorrect	Vérifiez le chemin dans .env (absolu recommandé)
Port 4000 déjà utilisé	Autre processus	Changez PORT dans .env ou arrêtez l'autre processus

6. Architecture technique

AutoRH repose sur une architecture Next.js 16 moderne avec un schéma de base de données Prisma (SQLite en local, compatible PostgreSQL/MySQL pour un déploiement serveur). Le frontend utilise React 19, Tailwind CSS 4 et shadcn/ui pour une interface responsive et accessible. L'authentification est gérée par des cookies httpOnly signés HMAC-SHA256, sans dépendance à un service externe (pas de NextAuth, pas de Firebase).

6.1. Stack technique

Composant	Technologie	Version
Framework	Next.js (Turbopack)	16.1
Base de données	Prisma + SQLite	6.12
UI	React + Tailwind CSS 4 + shadcn/ui	19 / 4 / latest
OCR	Tesseract.js + pdf.js	5 / 4
Canvas	@napi-rs/canvas	1.0
Signature	node-forge + openssl	1.3 / système

Composant	Technologie	Version
PDF	pdf-lib + ReportLab (génération)	1.17
Excel	xlsx (SheetJS)	0.18
Word	mammoth	1.6

6.2. Ports et services

Port	Service	URL d'accès
4000	Administration (RH/IT/SuperAdmin)	http://localhost:4000
4001	Portail employé (connexion CIN + mot de passe)	http://localhost:4001
3000	Pont d'aperçu (proxy reverse vers 4000)	http://localhost:3000

6.3. Structure des dossiers

```
auto-rh/ ├── src/ # Code source Next.js │ ├── app/ # Pages et API routes (App Router) │ │ ├── api/ # 50+ endpoints API REST │ │ ├── login/ # Page de connexion │ │ └── essai-expire/ # Page d'activation de licence (essai 15j) │ ├── lib/ # Modules serveur (auth, db, mailer, trial, etc.) │ ├── components/ # Composants React (UI) │ └── middleware.ts # Middleware (auth + trial) ├── prisma/schema.prisma # Schéma base de données ├── scripts/ # Scripts CLI (lanceurs, générateurs) ├── storage/ # Fichiers chiffrés (fiches, certificats) ├── public/ # Favicons, logo, assets statiques ├── .env # Configuration (secrets — JAMAIS commité) └── package.json
```